



操作系统实验报告

Lab 3: Programming with Threads

姓 名：李聪

学 号：22920212204124

院 系：信息学院人工智能系

专 业：人工智能

年 级：2021 级

指导教师：曹冬林

2023 年 12 月 28 日

一、实验目的	4
二、实验内容	4
2.1 实现事件屏障原语(EventBarrier Primitive)(15 分)	4
2.2 实现闹钟原语(Alarm Clock Primitive)(15 分)	4
2.3 同步问题 1: 桥(30 分)	5
2.4 同步问题 2: 电梯(40 分)	5
三、实验流程	6
3.1 实现事件屏障原语(EventBarrier Primitive)	6
3.1.1 编写 EventBarrier.h 和 EventBarrier.cc 文件	6
3.1.2 修改 threadtest.cc 文件	9
3.1.3 运行测试	9
3.2 实现闹钟原语(Alarm Clock Primitive)	9
3.2.1 编写 Alarm.h 和 Alarm.cc 文件	9
3.2.2 修改 threadtest.cc 文件	11
3.2.3 运行测试	11
3.3 同步问题 1: 桥(30 分)	11
3.3.1 编写 Bridge.cc 文件	11
3.3.2 修改 threadtest.cc 文件	13
3.3.3 运行测试	13
3.4 同步问题 2: 电梯(40 分)	13
3.4.1 编写 Elevator.h 和 Elevator.cc 文件	13
3.4.2 修改 threadtest.cc 文件	14
3.4.3 运行测试	14
四、测试分析	14
4.1 验证 EventBarrier 实现的正确性	14
4.1.1 运行 test1 函数	14
4.1.2 运行 test2 函数	15
4.2 验证 Alarm 实现的正确性	15
4.2.1 运行 test3 函数	15
4.3 验证桥问题实现的正确性	16
4.3.1 运行 test4 函数	16
4.4 验证电梯调度实现的正确性	16
4.4.1 运行 test6 函数	16
五、实验总结	17
5.1 EventBarrier 实现错误	17
六、实验心得	17
七、附录	17
附件 1 EventBarrier.h	17
附件 2 EventBarrier.cc	18
附件 3 Alarm.h 文件	20
附件 4 Alarm.cc 文件	20
附件 5 Bridge.cc 文件	22

附件 6 Elevator.h 文件	23
附件 7 Elevator.cc 文件	25
附件 8 threadtest.cc 文件	32
附件 9 test2 的样例输出	39
附件 10 test5 的样例输出	39
附件 11 test6 的样例输出	40

一、实验目的

本次实验的目的在于掌握使用 nachos 中的线程编程解决较为复杂的并发问题。

二、实验内容

2.1 实现事件屏障原语(EventBarrier Primitive)(15 分)

使用 Nachos 同步原语创建 EventBarrier 类，该类允许一组线程等待事件并以同步方式响应事件。EventBarrier 包含来自条件变量和信号量的有用属性。与条件变量一样，EventBarrier 与事件（或条件）相关联；线程可以等待事件或发出事件已经发生的信号。与二进制信号量一样，EventBarrier 不需要外部同步，它保持内部状态以“记住”先前的信号；如果事件已经发出信号，则等待立即返回。与条件变量或信号量不同，有信号的 EventBarrier 保持有信号状态，直到所有线程都响应了该信号，然后才恢复到无信号状态。为了确保所有线程都有机会响应，它创建了一个“屏障”，阻止信号线程和响应线程，直到所有参与的线程完成对事件的响应。这种行为使 EventBarrier 成为线程协调的强大原语。

这是 EventBarrier 的接口：

- void Wait() -- 等待直到事件发出信号。如果已经处于信号状态，立即返回。
- void Signal() -- 向事件发出信号并阻塞，直到等待此事件的所有线程都响应。当 Signal()返回时，EventBarrier 将恢复为无信号状态。
- void Complete() -- 指示调用线程已完成对信号事件的响应，并阻塞，直到等待此事件的所有其他线程也响应。
- int Waiters() -- 返回正在等待事件或尚未响应事件的线程计数。

注意：尽管名称不同，但 EventBarrier::Signal 更类似 Condition::Broadcast，而不是 Condition::Signal，因为 EventBarrier::Signal 会唤醒等待事件的所有线程。

您可以使用互斥对象、条件变量和/或信号量的任何组合来实现 EventBarrier，但是不要拘泥于禁用中断。以您认为最好的方式测试您的 EventBarrier 实现。确保您的实现正确地处理了在 EventBarrier 对象处于有信号状态时，调用 Wait 的线程；所有参与的线程必须等待，直到延迟到达响应事件并调用 Complete。另外，您的实现应该处理这样的情况，即线程在从 Complete 返回之后立即再次调用 Wait；这些线程应该在 Wait 中阻塞，直到 EventBarrier 再次处于有信号状态为止。

2.2 实现闹钟原语(Alarm Clock Primitive)(15 分)

实现一个 AlarmClock 类。线程调用 Alarm::Pause(int howLong)以进入睡眠一段时间。报警时钟可以参考 Timer(参考 Timer.h)来实现。当计时器中断响起时，system.cc 中 Timer 中断处理程序必须唤醒睡眠在 Alarm::Pause 中且间隔已经过期的任何线程。没有要求被唤醒的线程在间隔过期后立即开始运行；只需在它们等待至少指定的间隔(howLong)之后将它们放到就绪队列中。我们还没有为 Alarm 创建头文件，因此您可以根据需要定义类接口的其余部分。您可以为 howLong 使用任何方便的单位。

警告：不要更改计 **Timer** 硬件的行为以实现 **Alarms**。你可以修改 **Timer** 的中断处理程序 (**interrupt handler**)，但不要修改 **Timer** 类本身。为实现创建新的(模拟的)硬件 **Timer** 设备也是不可接受的。

警告：如果没有可运行的线程，即使有一个线程在等待警报 (**alarms**)，**Nachos** 也会退出。你必须想办法阻止这一切发生。这是一种罕见的情况，在这种情况下，“正确”的解决方案是修改 **machine** 子目录中的代码。欢迎您在这种情况下修改 **machine**，但它不是必需的。如果您不修改 **machine**，您将需要设计一个 **hack**，以防止 **Nachos** 退出时，有线程等待警报。一个丑陋但便宜的修复是分叉一个在紧密的循环中不断让步的线程，如果至少有一个线程等待警报。

警告：中断处理程序休眠是不正确的。考虑一下这个约束对同步方案的影响。此外，您应该设计您的代码，以尽量减少必须在中断时完成的工作量。

警告：除非使用 **-rs** 选项，否则 **Nachos** 不会提供计时器中断。您可以使用 **-rs** 选项，但最好调整初始化代码，以便在使用或不使用 **-rs** 时都具有正确的行为。

2.3 同步问题 1：桥(30 分)

你被 **MTA** 雇佣来同步公共高速公路上一座狭窄的轻型桥上的交通。车辆一次只能从一个方向通过，如果同时有超过 3 辆车在桥上，桥就会被它们的重量压垮。在这个系统中，每辆车由一个线程表示，当它到达桥时执行过程 **OneVehicle**：

```
OneVehicle( int direc ) {  
    ArriveBridge(direc);  
    CrossBridge(direc);  
    ExitBridge(direc);  
}
```

在上面的代码中，**direct** 是 0 或 1：它给出了车辆过桥的方向。

编写过程 **ArriveBridge** 和 **ExitBridge** (**CrossBridge** 过程应该只输出一条调试消息)，使用您选择的任何同步方案，包括互斥体、条件变量和/或信号量。**ArriveBridge** 必须在汽车可以安全通过给定方向的桥梁之前不返回(它必须保证不会发生正面碰撞或桥梁倒塌)。调用 **ExitBridge** 来指示调用者已经完成过桥，并且应该采取措施让其他车辆过桥。为了测试您的程序，创建一定数量的车辆，它们在不同的方向上重复使用桥梁：使用您的闹钟实现来模拟车辆的非桥梁活动。

您应该尝试使您的解决方案公平和无饥饿。为了理解这涉及到什么，试着回答以下关于你的方案的问题：一辆车到达时，车辆正沿着其行驶方向过桥，但已经有另一辆车在相反方向等待过桥，新到达的车是在另一边等待的车之前过，还是在另一边的车之后过，还是不能确定？

2.4 同步问题 2：电梯(40 分)

使用线程和您选择的同步方案(包括互斥锁、条件变量和/或信号量)，为具有 **F** 层的建筑物实现一个电梯控制器。你可能会发现你在这里实现的 **EventBarrier** 原语很有用。

电梯控制器分为两个类，电梯和建筑。我们已经在 **elevator.h** 中提供了这些类的框架定义。这些定义包括两组方法：一组用于内部使用，另一组用于定义调用方的外部接口。您需要向这些类添加新的内部方法，以允许在 **Elevator** 和 **Building** 类之间进行通信，但不要修改已经定义的方法的签名。

使用电梯的驱动程序将首先创建 **Building** 的单个实例，该实例将依次为大

楼中的每个电梯创建一个 `Elevator` 对象。每个电梯作为一个单独的线程运行。每个电梯线程应该在 `Elevator` 方法中进入一个无限循环，然后使用内部电梯控制接口(`OpenDoors`, `CloseDoors`, `VisitFloor` 和 `Building` 方法)以有序的方式满足乘客的请求。

在创建建筑之后，测试程序创建一个或多个 `riders`，每个 `riders` 作为一个单独的线程运行。乘客使用电梯乘客接口(`CallUp`、`CallDown`、`AwaitUp`、`AwaitDown`、`Enter`、`Exit`、`RequestFloor`)从电梯及其控制器请求服务。这些附加方法使用 `Elevator` 类和 `Building` 类中的共享状态与控制器方法同步。

您将实现这些类。注：没有负责 `building` 的线程；它的代码只在电梯或 `rider` 线程调用它的一个方法时执行。

第 1 部分。

为单个电梯实现电梯控制器，包括在 `elevator.h` 中定义的 `Building` 和 `Elevator` 方法，以及实现所需的任何新方法。你的解决方案应该避免乘客饥饿以及所有明显的竞争，例如，电梯在运输过程中门打开，电梯门关闭时乘客进出等。你可以假设电梯的大小是无界的，也就是说，它可以容纳所有到达的乘客。当你的电梯停在一层时，它应该等到所有离开的乘客都已经离开，所有上车的乘客都已经上车。

包括一个乘客测试程序，向多名乘客演示电梯的操作。您的乘客程序应该在以下意义上“表现良好”：任何呼叫电梯向上或向下方向的乘客都应该立即等待，直到电梯到达所要求的方向，然后登上电梯，然后请求在正确的方向上的楼层，然后在正确的楼层下车。使用闹钟实现对乘客的非电梯活动建模。

第 2 部分。

将第 1 部分中的解决方案扩展为处理一次只能容纳 `N` 个乘客的电梯，其中 `N` 是编译时常数。在本例中，如果乘客试图在电梯已满时上车，则 `Enter` 原语应该返回一个失败状态。

(额外学分:20 分):将你的解决方案扩展到第二部分，模拟一个有 `K(>1)` 部电梯的建筑物。

您将需要提出一个方案来决定使用 `k` 个电梯中的哪一个来处理请求。

警告:许多团队发现这个问题非常困难。编码前要三思。考虑解决方案的几个可能框架，然后选择最好的一个。争取一个简单而优雅的解决方案，即使它在性能方面不是最优的。

三、实验流程

3.1 实现事件屏障原语(EventBarrier Primitive)

3.1.1 编写 [EventBarrier.h](#) 和 [EventBarrier.cc](#) 文件

`EventBarrier` 类主要有三个接口 `Wait`、`Signal` 和 `Complete`。`EventBarrier` 主要作用是使线程可以“同时”完成对某件事的响应，比如说一起等电梯的乘客在都进入电梯后，电梯才可以继续运行。

具体来说，如下图 3.1.1 所示，有一些线程调用 `Wait` 等待某一事件的发生，会阻塞于 `signaled`。某一时刻，事件发生了，事件线程会调用 `Signal`，设置事件屏障处于有信号状态，唤醒阻塞于 `signaled` 的线程，并使后续调用 `Wait` 的线程不再阻塞于 `signaled`，可以直接响应事件。而最终这些线程都会调用 `Complete` 阻塞于 `complete`，等待所有线程完成对事件的响应，并由最后一个响应者唤醒 `Signal` 线程，由 `Signal` 线程发出全部完成响应的信号，唤醒所有

线程。

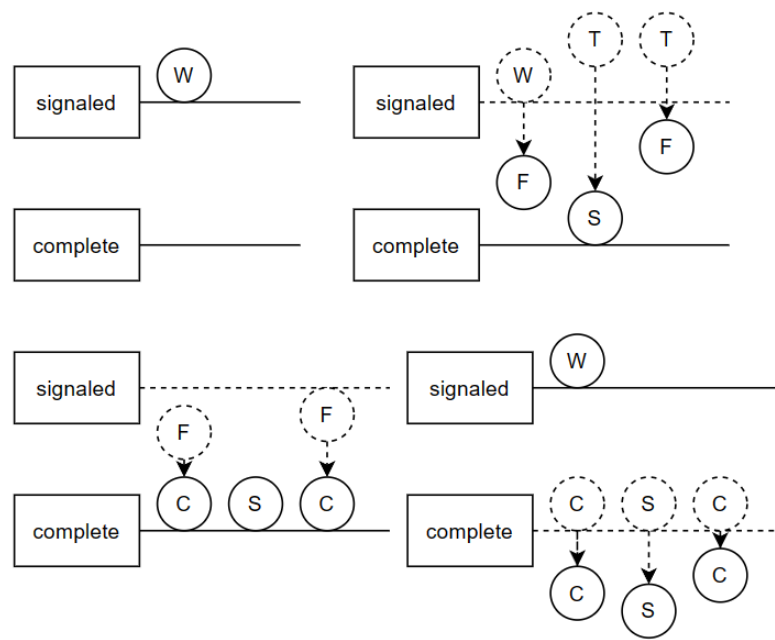


图 3.1.1

基于上述想法，设计了一个锁和三个条件变量，来分别等待事件信号的、完成信号和所有线程都完成的信号。

1) EventBarrier 类的定义如下

```
class EventBarrier
{
public:
    EventBarrier(char* debugName);
    ~EventBarrier();
    char* getName(){ return name; }
    void Wait();    // 等待事件发生的信号，若处于有信号状态，直接返回
    void Signal();  // 广播事件发生的信号，并等待所有线程都完成响应
                    // 当 Signal()返回时，EventBarrier 将恢复为无信号状态
    void Complete(); // 线程完成对事件信号的响应，并等待其他线程都完成响应
    int Waiters();   // 返回正在等待事件或尚未完成响应的线程的数量
private:
    char* name;
    int waiterNum;    // 正在等待事件或尚未完成响应的线程的数量
    bool signalStatus; // 有无信号状态
    Lock* lock;
    Condition* signaled;    // 有信号状态的条件变量
    Condition* complete;    // 完成响应的条件变量
    Condition* allCompleted; // 全部完成响应的条件变量
};
```

2) EventBarrier 的实现

A. 构造函数的实现

初始化 EventBarrier 类的成员变量，刚开始时，等待或尚未响应的线程数

量 waiterNum 为 0; EventBarrier 也处于无信号状态, 即 signalStatus 为 false。

```
EventBarrier::EventBarrier(char* debugName)
{
    name = debugName;
    waiterNum = 0;
    signalStatus = false;
    lock = new Lock("EventBarrier Lock");
    signaled = new Condition("Signaled Condition");
    complete = new Condition("Complete Condition");
    allCompleted = new Condition("AllComplete Condition");
}
```

B. Wait 的实现

Wait 的功能是检测是否处于有信号状态, 若处于无信号状态, 则阻塞等待事件信号, 反之, 直接返回并统计尚未完成响应的线程个数。具体过程如下:

- 递增 waiterNum;
- 判断是否处于有信号状态, 则直接返回;
- 执行 signaled->Wait(lock), 阻塞并等待事件信号。

```
void EventBarrier::Wait()
{
    lock->Acquire();
    ++ waiterNum;
    if(signalStatus)
    {
        lock->Release();
        return ;
    }
    signaled->Wait(lock);
    lock->Release();
}
```

C. Signal 的实现

Signal 的功能是发出事件信号, 唤醒等待事件信号的线程, 并阻塞等待所有线程完成对事件的响应。具体过程是:

- 将 signalStatus 赋值为 true, 表示处于有信号状态;
- 执行 eventSignal->Broadcast(lock)唤醒所有等待事件信号的线程;
- 当存在尚未完成响应的线程, 即 waiterNum>0 时, 执行 allCompleted->Wait(lock)等待所有线程完成响应, 直到所有线程完成响应;
- 执行 allCompleted->Wait(lock)唤醒所有等待全部线程完成响应的线程;
- 将 signalStatus 赋值为 false。

```
void EventBarrier::Signal()
{
    lock->Acquire();
    signalStatus = true;
    signaled->Broadcast(lock);
}
```



```

while(waiterNum > 0)
{
    allCompleted->Wait(lock);
}
complete->Broadcast(lock);
signalStatus = false;
lock->Release();
}

```

D. Complete 的实现

Complete 的功能是等待所有线程完成响应，并在所有线程完成响应后唤醒 Signal 线程。具体过程如下：

- a. 线程完成对线程的响应，waitNum 递减；
- b. 判断所有线程是否都完成响应，若都完成响应，则唤醒 Signal 线程；
- c. 执行 complete->Wait(lock)，等待所有线程完成响应，被 Signal 唤醒。

```

void EventBarrier::Complete()
{
    lock->Acquire();
    -- waiterNum;
    if(waiterNum == 0) allCompleted->Broadcast(lock);
    complete->Wait(lock);
    lock->Release();
}

```

e. Waiters 的实现

直接返回 waiterNum。

```

int EventBarrier::Waiters()
{
    return waiterNum;
}

```

3.1.2 修改 [threadtest.cc](#) 文件

1) 添加 [test1](#) 函数，可以创建三种线程：调用 Signal；先后调用 Wait 与 Complete；先后调用 Wait、Complete 与 Wait。用来验证特定的创建顺序下线程的行为的正确性。

2) 添加 [test2](#) 函数，可以创建两种线程：调用 Signal；先后调用 Wait 与 Complete。这些线程都先调用 alarm->Pause() 随机休眠一段时间，用来验证在更为随机的环境下 EventBarrier 实现的正确性。

3.1.3 运行测试

3.2 实现闹钟原语(Alarm Clock Primitive)

3.2.1 编写 [Alarm.h](#) 和 [Alarm.cc](#) 文件

Alarm 的实现是仿造 Timer 类的实现，利用 interrupt->Schedule() 函数像机器预定中断请求，在相应的时间可以会调用相应的中断处理程序来处理中断。

系统函数 void Interrupt::Schedule(VoidFunctionPtr handler, int arg, int fromNow, IntType type) 函数有四个参数，handler 是中断处理函数指针，arg 是中断处理函数的参数，fromNow 表示从现在起经过多久产生中断，type 是中断的类型。所以，在设计 Alarm::Pause 函数时，可以调用

这个函数预定一个中断请求后，进入休眠，在中断请求发生后，调用设计好的中断处理程序，唤醒这个线程，就能实现闹钟的效果。

下面将简单介绍一下，系统时刻是如何推进的。首先系统的时刻是以 **Tike** 为单位的，系统执行 **Interrupt::OneTick** 函数时，根据系统的状态推进时间，比如说我们一般处于 **SystemMode**，所以每次调用当前时刻会增加 **SystemTick**，即 **10 Tike**，而且在推进时间后，还会检测中断请求队列中，是否有中断请求该发生了，若有，则调用相应的中断处理函数。而 **Interrupt::OneTick** 调用是在 **Interrupt::SetLevel** 中，当中断标志从禁用变为启用时发生的。

1) **Alarm** 类的定义如下

```
class Alarm
{
public:
    Alarm();
    ~Alarm();
    void Pause(int howLong); // 睡 howLong Tick
    void Restart();          // 唤醒
private:
    int alarmNum; // 等待苏醒的线程数
    List* queue;  // 按苏醒时间顺序，插入线程
};
```

2) **Alarm** 类的实现

A. 构造函数的实现

初始化 **Alarm** 类的成员变量，刚开始休眠的线程数量为零，即 **alarmNum** 赋值为 **0**；并创建一个链表用来保存休眠线程的指针。

```
Alarm::Alarm()
{
    alarmNum = 0;
    queue = new List;
}
```

B. **Pause** 的实现

Pause 函数的功能是使调用线程休眠 **howLong Tick**。具体过程如下：

- 执行 **interrupt->Schedule(AlarmHandler, (int)this, howLong, TimerInt)**向系统预定一个中断请求；
- 将线程插入休眠队列；
- alarmNum** 计数加一；
- 线程进入休眠，等待。

```
void Alarm::Pause(int howLong)
{
    IntStatus oldLevel = interrupt->SetLevel(IntOff);
    if(alarmNum == 0)
    {
        Thread *thread = new Thread("Alarm");
        thread->Fork(hold, (int)&alarmNum);
    }
}
```

```

interrupt->Schedule(AlarmHandler, (int)this, howLong, TimerInt);
queue->SortedInsert(currentThread, stats->totalTicks + howLong);
++ alarmNum;
currentThread->Sleep();
(void)interrupt->SetLevel(oldLevel);
}

```

特别地，在让第一个休眠线程进入休眠前，需要创建一个不断执行线程切换的线程，防止系统因为所有线程进入休眠而直接退出。该线程调用的 `hold` 函数如下：

```

static void hold(int alarmNum)
{
    int* p = (int *) alarmNum;
    currentThread->Yield();
    while(*p)
    {
        currentThread->Yield();
    }
}

```

C. Restart 的实现

`Restart` 的作用是对应中断产生时，将线程从休眠队列中取走，放入就绪队列，并让 `alarmNum` 减一。

```

void Alarm::Restart()
{
    int when;
    IntStatus oldLevel = interrupt->SetLevel(IntOff);
    Thread* thread = (Thread*)queue->SortedRemove(&when);
    scheduler->ReadyToRun(thread);
    -- alarmNum;
    (void)interrupt->SetLevel(oldLevel);
}

```

需要注意的是 `Schedule` 的参数，只能传入具有一个 `int` 参数的函数指针，不能直接传入成员函数，所以需要编写一个 `AlarmHandler` 函数，将 `alarm` 对象强制类型转化为 `int` 作为函数，在函数中在转化为类对象指针调用 `Restart`。

```

static void AlarmHandler(int arg)
{
    Alarm *p = (Alarm *)arg;
    p->Restart();
}

```

3.2.2 修改 [threadtest.cc](#) 文件

1) 添加 [test3](#) 函数，可以创建线程随机生成休眠时间，并执行 `alarm->Pause(howLong)` 进入休眠。用来验证 `Alarm` 实现的正确性。

3.2.3 运行测试

3.3 同步问题 1: 桥(30 分)

3.3.1 编写 [Bridge.cc](#) 文件

车的行为主要是三个函数 **ArriveBridge**、**CrossBridge** 和 **ExitBridge** 分别表示抵达桥头、过桥和离开桥。问题规定了桥的最大通行数量，以及桥是单向通行的。为了避免饥饿，采用先来先服务的原则，即先到的车先过桥，为此需要设计两个条件变量，一个用来保证最大通行数量，另一个用来保证最多只用一个车在检测能否过桥。用 **direction** 表示桥的方向，**carInBridge** 表示桥上彻底数量，**checkNum** 表示正在检测能否过桥并等待的车的数量。

1) ArriveBridge 的实现

当有别的车已经在等待过桥了，即 **checkNum** 大于 1，车辆线程会被阻塞在 **check** 等待前面的车都通过；当车在要过桥时，重复判断是否可通行，直到可通行，否则执行 **passable->Wait(lock)** 被阻塞，可通行的条件是车的方向和桥的方向相同且桥上车数量小于三，或者桥上没有车。完成过桥的准备后，可以通知后来的车准备过桥，且根据方向适当地更改桥的方向，以及桥上车数量。

```
bool IsPassable(int direc)
{
    if(carInBridge == 0 || (direc == direction && carInBridge < 3))
    {
        return true;
    }
    else
    {
        return false;
    }
}

void ArriveBridge(int direc)
{
    lock->Acquire();
    while(checkNum)
    {
        check->Wait(lock);
    }
    ++ checkNum;
    while(!IsPassable(direc))
    {
        passable->Wait(lock);
    }
    -- checkNum;
    check->Signal(lock);
    direction = direc;
    ++ carInBridge;
    lock->Release();
}
```

2) CrossBridge 的实现

输出一条调试语句，表示过桥时间。

```
void CrossBridge(int direc)
```

```
{
    DEBUG('e', "%s cross the bridge, direction:%d.\n",
currentThread->getName(), direc);
}
```

3) ExitBridge 的实现

车过桥后，应该更新桥上车数量，以及唤醒等待通过桥的车。

```
void ExitBridge(int direc)
{
    lock->Acquire();
    -- carInBridge;
    passable->Signal(lock);
    lock->Release();
}
```

3.3.2 修改 [threadtest.cc](#) 文件

1) 添加 [test4](#) 函数，可以创建两种方向的车，并按照特定顺序抵达桥头。用来验证成功消除饥饿。

2) 添加 [test5](#) 函数，可以创建汽车线程，随机方向，随机抵达时间。用来验证实现的正确性。

3.3.3 运行测试

3.4 同步问题 2：电梯(40 分)

3.4.1 编写 [Elevator.h](#) 和 [Elevator.cc](#) 文件

1) 乘客的行为：

a. 在某一楼层按下呼叫电梯的按钮，等待电梯的到来[**Await**]，由 **building** 处理这个请求，为乘客分配电梯。

b. 等待的电梯抵达时，尝试进入电梯[**Enter**]，若电梯满了，则回到 1，继续请求电梯。

c. 乘客登上电梯后，向电梯发送请求到目标楼层，并等待电梯运行到对应楼层[**RequestFloor**]。

d. 出电梯[**Exit**]。

2) 电梯的行为：

a. 运行 **Run**：先判断电梯内是否无人，若无人，则向 **building** 发送可以载客的信号；然后获取最近的目标楼层，若没有，则等待乘客通过 **building** 发送的请求。

b. 开门 **OpenDoors**：先通知电梯内的乘客出电梯，然后通知电梯外要进电梯的进来。

c. 关门 **CloseDoorsS**：输出一句调试信息

3) **building** 的行为：

building 会接受来自乘客的请求，然后为这个乘客分配一个电梯，分配的方法是从上一个分配的电梯开始，以此检测，所有的电梯，是否能满足这个请求，满足这个条件下面三个条件中的一个：

a) 电梯无请求

b) 电梯中无人，在去接其他乘客的路上，此时乘客方向应该与电梯接客后将运行的方向相同且在这个运行路径上。比如说电梯从 1 层到 8 层接客，然后继续向上，则比当前楼层高的向上的请求可以响应。

c) 电梯中有人，应该与此时电梯运行的方向一致，且在运行路径方向上。

如果说，所有电梯都不能满足，则等待某个电梯空闲，发出信号，重新判断请求是否可满足的。

3.4.2 修改 [threadtest.cc](#) 文件

1) 添加 [test6](#) 函数，可以创建电梯和乘客线程，根据命令行参数可以指定楼层、电梯的个数、电梯的容量、乘客的数量，乘客线程可以随机生成请求，以及随机抵达时间。用来验证电梯调度实现的正确性。

3.4.3 运行测试

四、测试分析

4.1 验证 `EventBarrier` 实现的正确性

4.1.1 运行 [test1](#) 函数

命令： `./nachos -q 1 -d e`

该测试验证 `EventBarrier` 特定调用顺序下，是否符合预期。

输出样例如下：

```
1 $ ./nachos -q 1 -d e
2 Test 0: call Signal.
3 Test 0: finish.
4 Test 1: call Wait.
5 Test 2: call Wait.
6 Test 3: call Wait.
7 Test 4: call Signal.
8 Test 5: call Wait.
9 Test 5: call Complete.
10 Test 6: call Wait.
11 Test 6: call Complete.
12 Test 7: call Wait.
13 Test 7: call Complete.
14 Test 8: call Wait.
15 Test 8: call Complete.
16 Test 1: call Complete.
17 Test 2: call Complete.
18 Test 3: call Complete.
19 Test 4: finish.
20 Test 5: call Wait.
21 Test 6: finish.
22 Test 7: finish.
23 Test 8: call Wait.
24 Test 1: call Wait.
25 Test 2: call Wait.
26 Test 3: finish
```

分析：

`Test 0` 调用了 `Signal`，因为一开始没有等待响应事件的线程，所以 `Test`

0 直接从 **Signal** 中返回，结束线程。

在 **Test 4** 调用 **Signal**，即事件信号到来之前，**Test 1~3** 调用 **Wait** 会被阻塞，而在 **Test 4** 之后的线程 **Test 5~8**，调用 **Wait** 会直接返回，完成对事件的响应，调用 **Complete**，并阻塞等待所有线程完成响应。

Test 1~3 被 **Test 4** 成功唤醒，并完成对事件的响应，调用 **Complete** 阻塞并等待线程完成响应。

在所有线程完成响应后，由最后一个 **Complete** 调用者 **Test 8**，唤醒 **Test 4**，自身则被阻塞，最后由 **Test 4** 唤醒所有的线程。

可以观察到并不是所有的线程都会 **finish**，这是因为 **Test 5、8、1、2** 在 **Complete** 返回后，又调用了 **Wait** 被阻塞，这符合题目的描述。

4.1.2 运行 **test2** 函数

命令： `./nachos -q 2 n1 n2 t -d e`

该测试调用 **test2** 函数，参数如下：

- **n1**--调用 **Signal** 的线程的数量
- **n2**--调用 **Wait** 和 **Complete** 的线程的数量
- **t**--线程最大休眠时间

重复运行多次，均符合预期。典型测试样例见[附件 9](#)。

4.2 验证 **Alarm** 实现的正确性

4.2.1 运行 **test3** 函数

命令： `./nachos -q 3 n t -d s`

该测试调用 **test3** 函数，参数如下：

- **n**--线程的数量
- **t**--线程的最大休眠时间

重复运行多次，均符合预期，没有出现错误。

输出样例如下：

```
1 $ ./nachos -q 3 5 1000 -d s
2 main: make thread AlarmTest 0
3 main: make thread AlarmTest 1
4 main: make thread AlarmTest 2
5 main: make thread AlarmTest 3
6 main: make thread AlarmTest 4
7 AlarmTest 0: sleep 753 tick from Tick 70 to Tick 823.
8 AlarmTest 1: sleep 854 tick from Tick 80 to Tick 934.
9 AlarmTest 2: sleep 251 tick from Tick 90 to Tick 341.
10 AlarmTest 3: sleep 736 tick from Tick 100 to Tick 836.
11 AlarmTest 4: sleep 145 tick from Tick 110 to Tick 255.
12 AlarmTest 4: wakeup in Tick 260.
13 AlarmTest 2: wakeup in Tick 350.
14 AlarmTest 0: wakeup in Tick 830.
15 AlarmTest 3: wakeup in Tick 840.
16 AlarmTest 1: wakeup in Tick 940.
```

分析：

可以看到创建了 5 个线程，随机休眠一段时间，醒来的时间是预计醒来时间的上取整十，这是因为我们运行在 **SystemMode**，时间刻的推进是以 10 Tike 为

单位的。

4.3 验证桥问题实现的正确性

4.3.1 运行 [test4](#) 函数

命令: `./nachos -q 4 -d e`

该测试函数调用了 `test4`，按照特殊顺序创建两种不同方向的汽车线程，来观察特定抵达顺序下的，汽车通过桥的顺序，是否会发生饥饿。

输出样例如下：

```
1 $ ./nachos -q 4 -d e
2 Car 0: arrive 0.
3 Car 0: enter bridge.
4 Car 1: arrive 0.
5 Car 1: enter bridge.
6 Car 2: arrive 1.
7 Car 3: arrive 0.
8 Car 0 cross the bridge, direction:0.
9 Car 1 cross the bridge, direction:0.
10 Car 2: enter bridge.
11 Car 2 cross the bridge, direction:1.
12 Car 3: enter bridge.
13 Car 3 cross the bridge, direction:0.
```

分析：

可以看到 Car 3 到达时，Car 0 和 Car 1 正沿着 Car 3 行驶方向过桥，但已经有另一辆车 Car 2 在相反方向等待过桥，而 Car 3 是在 Car 2 通过桥之后，再过桥的，这符合先来先服务的设计预期，很好地解决了饥饿的问题。

4.3.2 运行 [test5](#) 函数

命令: `./nachos -q 5 n t -d e`

该测试函数调用 `test5` 函数，参数如下：

- `n`--汽车线程的数量
- `t`--汽车抵达桥的最大时间

重复运行多次，均符合预期。典型测试样例见[附件 10](#)。

4.4 验证电梯调度实现的正确性

4.4.1 运行 [test6](#) 函数

命令: `./nachos -q 6 f e c n t -d e`

该测试函数调用 `test6` 函数，参数如下：

- `f`--楼层数
- `e`--电梯数量
- `c`--电梯容量
- `n`--乘客的数量
- `t`--乘客最大的抵达时间

多次运行不同电梯数量，不同容量下的调度程序，结果均符合预期。

输出样例：

```
$ ./nachos -q 6 5 1 0 1 100 -d e
Rider 0: want to go from 1 to 4.
Elevator 0: moving from 0 to 1.
```


Elevator 0: arrive 1.
Elevator 0: open doors.
Rider 0: enter Elevator 0 in Floor 1.
Elevator 0: close doors.
Rider 0: request to go to 4.
Elevator 0: moving from 1 to 2.
Elevator 0: arrive 2.
Elevator 0: moving from 2 to 3.
Elevator 0: arrive 3.
Elevator 0: moving from 3 to 4.
Elevator 0: arrive 4.
Elevator 0: open doors.
Rider 0: exit Elevator 0 in Floor 4.
Elevator 0: close doors.
Rider 0: finished.

分析：

上面的输出信息，展示了一个电梯和一个乘客的行为，乘客想要从 1 层到 4 层，按下按钮，电梯调度程序会为他分配一个电梯。电梯在接收到请求后，会被唤醒，开始从第 0 层运行到 1 层。电梯抵达 1 层后，会打开门。乘客看到电梯来了，会进入电梯。电梯关门。然后乘客告诉电梯目标楼层 4 层。电梯继续运行从 1 层运行到 4 层。电梯抵达 4 层后，打开门。乘客出电梯。电梯关门。

多电梯典型测试样例见[附件 11](#)。

五、实验总结

5.1 EventBarrier 实现错误

最开始理解的 EventBarrier 实现是由最后一个调用 Complete 函数的线程，执行 complete->Broadcast()唤醒所有线程，但是这样存在着一个问题，最后一个调用 Complete 函数的线程在返回后，立即调用 Wait 不会被阻塞，这种情况和题目中的描述不相符，所以改成了现在这个由最后一个调用 Complete 函数的线程唤醒 Signal 线程，让 Signal 线程判断是否真的都完成响应了，如果真的都完成响应了，Signal 会唤醒所有线程。

六、实验心得

在本次实验中，在了解了题目要求的情况下，成功实现了事件屏障线程同步工具以及闹钟原语，还解决了两个同步问题。通过本次实验也了解到了许多编程上的技巧，比如如何正确使用命令行参数，如何减少代码之间的冲突，以及如何 DEBUG。还有通过仔细查阅中断请求的相关文件，了解到了中断时如何产生的，以及如何处理中断。

七、附录

附件 1 EventBarrier.h

```
1 # ifndef EVENTBARRIER_H
2 # define EVENTBARRIER_H
3 # include "synch.h"
```

```

4  class EventBarrier
5  {
6  public:
7      EventBarrier(char* debugName);
8      ~EventBarrier();
9      char* getName(){ return name; }
10     void Wait();    // 等待事件发生的信号，若处于有信号状态，直接返回
11     void Signal();  // 广播事件发生的信号，并等待所有线程都完成响应
12                     // 当 Signal()返回时，EventBarrier 将恢复为无信号状态
13     void Complete(); // 线程完成对事件信号的响应，并等待其他线程都完成响应
14     int Waiters();  // 返回正在等待事件或尚未完成响应的线程的数量
15 private:
16     char* name;
17     int waiterNum;    // 正在等待事件或尚未完成响应的线程的数量
18     bool signalStatus; // 有无信号状态
19     Lock* lock;
20     Condition* signaled;    // 有信号状态的条件变量
21     Condition* complete;    // 完成响应的条件变量
22     Condition* allCompleted; // 全部完成响应的条件变量
23 };
24 # endif

```

附件 2 EventBarrier.cc

```

1  # include "EventBarrier.h"
2
3  EventBarrier::EventBarrier(char* debugName)
4  {
5      name = debugName;
6      waiterNum = 0;
7      signalStatus = false;
8      lock = new Lock("EventBarrier Lock");
9      signaled = new Condition("Signaled Condition");
10     complete = new Condition("Complete Condition");
11     allCompleted = new Condition("AllComplete Condition");
12 }
13
14 EventBarrier::~~EventBarrier()
15 {
16     delete lock;
17     delete signaled;
18     delete complete;
19     delete allCompleted;
20 }
21

```

```
22 // 等待事件发生的信号，若处于有信号状态，直接返回
23 void EventBarrier::Wait()
24 {
25     lock->Acquire();
26     ++ waiterNum;
27     if(signalStatus)
28     {
29         lock->Release();
30         return ;
31     }
32     signaled->Wait(lock);
33     lock->Release();
34 }
35
36 // 广播事件发生的信号，并等待所有线程都完成响应
37 // 当 Signal()返回时，EventBarrier 将恢复为无信号状态
38 void EventBarrier::Signal()
39 {
40     lock->Acquire();
41     signalStatus = true;
42     signaled->Broadcast(lock);
43     while(waiterNum > 0)
44     {
45         allCompleted->Wait(lock);
46     }
47     complete->Broadcast(lock);
48     signalStatus = false;
49     lock->Release();
50 }
51
52 // 线程完成对事件信号的响应，并等待其他线程都完成响应
53 void EventBarrier::Complete()
54 {
55     lock->Acquire();
56     -- waiterNum;
57     if(waiterNum == 0) allCompleted->Broadcast(lock);
58     complete->Wait(lock);
59     lock->Release();
60 }
61
62 // 返回正在等待事件或尚未完成响应的线程的数量
63 int EventBarrier::Waiters()
```

```
64 {
65     return waiterNum;
66 }
```

附件 3 Alarm.h 文件

```
1  # ifndef ALARM_H
2  # define ALARM_H
3  # include "list.h"
4  # include "synch.h"
5  class Alarm
6  {
7  public:
8      Alarm();
9      ~Alarm();
10     void Pause(int howLong); // 睡 howLong Tick
11     void Restart();          // 唤醒
12 private:
13     int alarmNum; // 等待苏醒的线程数
14     List* queue;  // 按苏醒时间顺序, 插入线程
15 };
16 # endif
```

附件 4 Alarm.cc 文件

```
1  #include "Alarm.h"
2  #include "system.h"
3
4  Alarm* alarm = new Alarm();
5
6  static void AlarmHandler(int arg)
7  {
8      Alarm *p = (Alarm *)arg;
9      p->Restart();
10 }
11
12 Alarm::Alarm()
13 {
14     alarmNum = 0;
15     queue = new List;
16 }
17
18 Alarm::~~Alarm()
19 {
20     delete queue;
21 }
22
```

```

23 static void hold(int alarmNum)
24 {
25     int* p = (int *) alarmNum;
26     currentThread->Yield();
27     while(*p)
28     {
29         currentThread->Yield();
30     }
31 }
32
33 void Alarm::Pause(int howLong)
34 {
35     IntStatus oldLevel = interrupt->SetLevel(IntOff);
36
37     if(alarmNum == 0)
38     {
39         Thread *thread = new Thread("Alarm");
40         thread->Fork(hold, (int)&alarmNum);
41     }
42
43     interrupt->Schedule(AlarmHandler, (int)this, howLong, TimerInt);
44     queue->SortedInsert(currentThread, stats->totalTicks + howLong);
45     ++ alarmNum;
46
47     DEBUG('s', "%s: sleep %d tick from Tick %d to Tick %d.\n",
48         currentThread->getName(), howLong, stats->totalTicks, stats->totalTicks
49 + howLong);
50
51     currentThread->Sleep();
52
53     (void)interrupt->SetLevel(oldLevel);
54 }
55
56 void Alarm::Restart()
57 {
58     int when;
59     IntStatus oldLevel = interrupt->SetLevel(IntOff);
60
61     Thread* thread = (Thread*)queue->SortedRemove(&when);
62     ASSERT(stats->totalTicks >= when);
63     scheduler->ReadyToRun(thread);
64     -- alarmNum;

```

```

65
66     DEBUG('s', "%s: wakeup in Tick %d.\n", thread->getName(),
67 stats->totalTicks);
68
69     (void)interrupt->SetLevel(oldLevel);
70 }

```

附件 5 Bridge.cc 文件

```

1  # include "synch.h"
2  # include "system.h"
3  # include "Alarm.h"
4
5  static int direction = 0;
6  static int carInBridge = 0;
7  static int checkNum = 0;
8  static Lock* lock = new Lock("Bridge Lock");
9  static Condition* check = new Condition("Check Condition");
10 static Condition* passable = new Condition("Passable Condition");
11
12 bool IsPassable(int direc)
13 {
14     if(carInBridge == 0 || (direc == direction && carInBridge < 3))
15     {
16         return true;
17     }
18     else
19     {
20         return false;
21     }
22 }
23
24 void ArriveBridge(int direc)
25 {
26     lock->Acquire();
27     while(checkNum)
28     {
29         check->Wait(lock);
30     }
31     ++ checkNum;
32     while(!IsPassable(direc))
33     {
34         passable->Wait(lock);
35     }
36     -- checkNum;

```

```

37     check->Signal(lock);
38     direction = direc;
39     ++ carInBridge;
40     lock->Release();
41 }
42
43 void CrossBridge(int direc)
44 {
45     DEBUG('e', "%s cross the bridge, direction:%d.\n",
46     currentThread->getName(), direc);
47 }
48
49 void ExitBridge(int direc)
50 {
51     lock->Acquire();
52     -- carInBridge;
53     passable->Signal(lock);
54     lock->Release();
55 }

```

附件 6 Elevator.h 文件

```

1  #ifndef ELEVATOR_H
2  #define ELEVATOR_H
3  #include "system.h"
4  #include "EventBarrier.h"
5  #include "copyright.h"
6  #include "synch.h"
7  #include "Alarm.h"
8  class Elevator
9  {
10 public:
11     Elevator(char *debugName, int numFloors, int id, int maxCapacity);
12     ~Elevator();
13     char *GetName() { return name; }
14
15     // 由电梯线程调用
16     void OpenDoors(); // 开门
17     void CloseDoors(); // 关门
18     void Run(); // 电梯运行
19     bool IsSatisfiable(int direc, int fromFloor); // 判断电梯是否可以响应这个
20  请求
21
22     // 由乘客线程调用

```

```

23     bool Enter();    // 进电梯
24     void Exit();    // 出电梯
25     void RequestFloor(int dstFloor);    // 告诉电梯乘客的目标楼层
26     void CallIn(int direc,int fromFloor);    // 呼叫并等待这个电梯的到来
27 private:
28     int GetNextFloor(); // 获得最近的目标楼层
29
30     char *name;        // 电梯名称
31     int elevatorId;    // 电梯 id
32     int num_floor;     // 电梯楼层数
33     int currentFloor;  // 电梯当前停靠的楼层
34     int direction;     // 电梯正在运行方向
35     int nextDirec;     // 暂存有乘客后的要运行的方向
36     int capacity;      // 电梯的最大容量
37     int number;        // 电梯当前乘客数量
38     EventBarrier **exitElevator;    // 各层等待进电梯的事件栅栏
39     EventBarrier **enterElevator;   // 各层等待出电梯的事件栅栏
40     Lock* elevatorLock;             // 电梯锁
41     Condition* request;             // 电梯存在请求的条件
42 };
43
44 class Building
45 {
46 public:
47     Building(char *debugName, int numFloors, int numElevators, int
48 maxCapacity);
49     ~Building();
50     char *getName() { return name; }
51     Elevator* getElevator(int id); // 获得特定的电梯
52     Elevator *Await(int direc, int fromFloor); // 呼叫并等待电梯的到来
53     void Signal(); // 发出存在电梯可以响应的信号
54 private:
55     Elevator *Call(int direc, int fromFloor); // 呼叫电梯
56
57     char *name;        // 建筑物名称
58     int num_floor;     // 楼层数量
59     int num_elevator;  // 电梯数量
60     Elevator **elevator; // 电梯列表
61     Lock* buildingLock; // 建筑锁
62     Condition* responsible; // 存在可响应电梯条件
63     int idx;
64 };

```


65 #endif

附件 7 Elevator.cc 文件

```
1 #include "Elevator.h"
2
3 extern Alarm* alarm;
4
5 Building *building;
6
7 void rider(int srcFloor, int dstFloor)
8 {
9     Elevator *e;
10    DEBUG('e', "%s: want to go from %d to %d.\n", currentThread->getName(),
11    srcFloor, dstFloor);
12
13    if (srcFloor == dstFloor)
14    {
15        DEBUG('e', "%s: \033[40;32mfinished\033[0m.\n",
16    currentThread->getName());
17        return;
18    }
19
20    do
21    {
22        if(srcFloor < dstFloor)
23        {
24            e = building->Await(1, srcFloor);
25        }
26        else if(srcFloor > dstFloor)
27        {
28            e = building->Await(-1, srcFloor);
29        }
30    } while (!e->Enter());
31    DEBUG('e', "%s: request to go to %d.\n", currentThread->getName(),
32    dstFloor);
33    e->RequestFloor(dstFloor);
34    e->Exit();
35    DEBUG('e', "%s: \033[40;32mfinished\033[0m.\n",
36    currentThread->getName());
37 }
38
39 void elevator(int id)
40 {
41     Elevator* elevator = building->getElevator(id);
```

```

42     while(true)
43     {
44         elevator->Run();
45         elevator->OpenDoors();
46         elevator->CloseDoors();
47     }
48 }
49
50 Elevator::Elevator(char *debugName, int numFloors, int id, int maxCapacity)
51 {
52     name = debugName;          // 电梯名称
53     elevatorId = id;           // 电梯 id
54     num_floor = numFloors;     // 电梯楼层数
55     currentFloor = 0;          // 电梯当前停靠的楼层
56     direction = 0;             // 电梯运行方向
57     capacity = maxCapacity;    // 电梯的最大容量
58     number = 0;                // 电梯当前乘客数量
59     exitElevator = new EventBarrier*[num_floor];    // 各层等待进电梯的事件栅
60 栏
61     enterElevator = new EventBarrier*[num_floor];  // 各层等待出电梯的事件栅
62 栏
63     elevatorLock = new Lock("Elevator Lock");     // 电梯锁
64     request = new Condition("Request Condition");  // 电梯存在请求的条件
65     for(int i = 0; i < num_floor; ++ i)
66     {
67         exitElevator[i] = new EventBarrier("ExitElevator EventBarrier");
68         enterElevator[i] = new EventBarrier("EnterElevator EventBarrier");
69     }
70 }
71
72 Elevator::~~Elevator()
73 {
74     delete elevatorLock;
75     delete request;
76     for(int i = 0; i < num_floor; ++ i)
77     {
78         delete exitElevator[i];
79         delete enterElevator[i];
80     }
81     delete exitElevator;
82     delete enterElevator;
83 }

```

```
84
85 // 开门
86 void Elevator::OpenDoors()
87 {
88     DEBUG('e', "%s: open doors.\n", currentThread->getName());
89     // 先下后上
90     exitElevator[currentFloor]->Signal(); // 通知乘客出电梯
91     DEBUG('s', "%s: signal enter Elevator.\n", currentThread->getName());
92     enterElevator[currentFloor]->Signal(); // 通知乘客进电梯
93 }
94
95 // 关门
96 void Elevator::CloseDoors()
97 {
98     DEBUG('e', "%s: close doors.\n", currentThread->getName());
99 }
100
101 // 电梯运行
102 void Elevator::Run()
103 {
104     DEBUG('s', "%s: in Run.\n", currentThread->getName());
105     if(number == 0) // 电梯无人，应处于空闲状态
106     {
107         direction = 0;
108         building->Signal();
109     }
110     elevatorLock->Acquire();
111     int dstFloor = GetNextFloor(); // 最近的目标楼层
112     while(dstFloor == -1) // 判断是否存在请求
113     {
114         request->Wait(elevatorLock); // 等待请求
115         dstFloor = GetNextFloor();
116     }
117     elevatorLock->Release();
118     while(currentFloor != dstFloor)
119     {
120         // 移动一层
121         ASSERT(direction != 0);
122         DEBUG('e', "%s: moving from %d to %d.\n",
123             currentThread->getName(), currentFloor, currentFloor +
124             direction);
125         alarm->Pause(50);
```

```
126         currentFloor += direction;
127         DEBUG('e', "%s: arrive %d.\n", currentThread->getName(),
128 currentFloor);
129         dstFloor = GetNextFloor(); // 寻找是否存在更近的目标楼层
130     }
131     if(number == 0) // 电梯接第一个人前，应该改变运行方向
132     {
133         direction = nextDirec;
134     }
135     DEBUG('s', "%s: out Run.\n", currentThread->getName());
136 }
137
138 // 进电梯
139 bool Elevator::Enter()
140 {
141     if(number < capacity || capacity == 0)
142     {
143         DEBUG('e', "%s: enter Elevator %d in Floor %d.\n",
144             currentThread->getName(), elevatorId, currentFloor);
145         ++ number;
146         enterElevator[currentFloor]->Complete();
147         return true;
148     }
149     else
150     {
151         // 该电梯满了，等下一个电梯
152         enterElevator[currentFloor]->Complete();
153         return false;
154     }
155 }
156
157 // 出电梯
158 void Elevator::Exit()
159 {
160     DEBUG('e', "%s: exit Elevator %d in Floor %d.\n",
161         currentThread->getName(), elevatorId, currentFloor);
162     -- number;
163     exitElevator[currentFloor]->Complete();
164 }
165
166 // 告诉电梯乘客的目标楼层
167 void Elevator::RequestFloor(int dstFloor)
```

```

168 {
169     DEBUG('s', "%s: in RequestFloor.\n", currentThread->getName());
170     elevatorLock->Acquire();
171     request->Signal(elevatorLock); // 发送存在请求的信号
172     elevatorLock->Release();
173     exitElevator[dstFloor]->Wait(); // 乘客等待电梯到达目标楼层
174     DEBUG('s', "%s: out RequestFloor.\n", currentThread->getName());
175 }
176
177 // 判断电梯是否可以响应这个请求
178 // 可响应条件：电梯无请求或电梯的可能运行路径与请求相符
179 bool Elevator::IsSatisfiable(int direc, int fromFloor)
180 {
181     if(direction == 0)
182     {
183         // 如果电梯无请求，那么肯定能响应请求
184         return true;
185     }
186     else if((number == 0 && direc == nextDirec) || (number > 0 && number <
187 capacity && direc == direction))
188     {
189         // 电梯内无人，说明电梯在去接客的路上，所以要和电梯接客后的方向一致
190         // 电梯内有人，只有电梯未满时，才能响应请求，且应该与运行方向一致
191         if(direc == 1) return fromFloor >= currentFloor;
192         else return fromFloor <= currentFloor;
193     }
194     return false;
195 }
196
197 // 呼叫并等待这个电梯的到来
198 void Elevator::CallIn(int direc, int fromFloor)
199 {
200     DEBUG('s', "%s: in CallIn.\n", currentThread->getName());
201     elevatorLock->Acquire();
202     if(direction == 0 && number == 0)
203     {
204         // 呼叫到的是一个无请求电梯
205         // 为电梯设定运行方向
206         if(currentFloor < fromFloor)
207         {
208             direction = 1;
209         }

```

```

210         else if (currentFloor > fromFloor)
211         {
212             direction = -1;
213         }
214         nextDirec = direc;
215         request->Signal(elevatorLock); // 发送存在请求的信号
216     }
217     elevatorLock->Release();
218     enterElevator[fromFloor]->Wait(); // 乘客等待入电梯
219     DEBUG('s', "%s: out CallIn.\n", currentThread->getName());
220 }
221
222 // 获得最近的目标楼层
223 int Elevator::GetNextFloor()
224 {
225     if(direction == 0)
226     {
227         if(exitElevator[currentFloor]->Waiters() ||
228 enterElevator[currentFloor]->Waiters())
229         {
230             return currentFloor;
231         }
232         else
233         {
234             return -1;
235         }
236     }
237     for(int i = currentFloor; i >= 0 && i < num_floor; i += direction)
238         if(exitElevator[i]->Waiters() || enterElevator[i]->Waiters())
239         {
240             return i;
241         }
242     return -1;
243 }
244
245 Building::Building(char *debugName, int numFloors, int numElevators, int
246 maxCapacity)
247 {
248     name = debugName;
249     num_floor = numFloors;
250     num_elevator = numElevators;
251     elevator = new Elevator*[num_elevator];

```

```

252     for(int id = 0; id < num_elevator; ++ id)
253         elevator[id] = new Elevator("Elevator", numFloors, id,
254 maxCapacity);
255     buildingLock = new Lock("Building Lock");
256     responsible = new Condition("Responsible Condition");
257     idx = 0;
258 }
259
260 Building::~Building()
261 {
262     delete buildingLock;
263     delete responsible;
264     for(int i = 0; i < num_elevator; ++ i)
265         delete elevator[i];
266     delete elevator;
267 }
268
269 // 呼叫电梯
270 Elevator *Building::Call(int direc, int fromFloor)
271 {
272     // 遍历每个电梯返回可满足请求的。若没有，返回 NULL。
273     for(int i = 0; i < num_elevator; ++ i)
274     {
275         int id = (idx + i) % num_elevator;
276         if(elevator[id]->IsSatisfiable(direc, fromFloor))
277         {
278             idx = id;
279             return elevator[id];
280         }
281     }
282     return NULL;
283 }
284
285 // 呼叫并等待电梯的到来
286 Elevator *Building::Await(int direc, int fromFloor)
287 {
288     DEBUG('s', "%s: in Await.\n", currentThread->getName());
289     buildingLock->Acquire();
290     // 重复呼叫电梯直到被响应
291     Elevator* elev = Call(direc, fromFloor);
292     while(elev == NULL)
293     {

```

```

294     responsible->Wait(buildingLock);
295     elev = Call(direc, fromFloor);
296 }
297
298 buildingLock->Release();
299 // 等待响应的电梯到来
300 elev->CallIn(direc, fromFloor);
301 DEBUG('s', "%s: out Await.\n", currentThread->getName());
302 return elev;
303 }
304
305 // 发出存在电梯可以响应的信号
306 void Building::Signal()
307 {
308     buildingLock->Acquire();
309     responsible->Signal(buildingLock);
310     buildingLock->Release();
311 }
312
313 Elevator *Building::getElevator(int id)
314 {
315     return elevator[id];
316 }

```

附件 8 threadtest.cc 文件

```

1 #include <stdlib.h>
2 #include <time.h>
3 #include "system.h"
4 #include "Alarm.h"
5 #include "Elevator.h"
6 #include "EventBarrier.h"
7
8 extern Alarm* alarm;
9 extern Building* building;
10 extern void ArriveBridge(int direc);
11 extern void CrossBridge(int direc);
12 extern void ExitBridge(int direc);
13 extern void rider(int srcFloor, int dstFloor);
14 extern void elevator(int id);
15
16 const int CrossBridgeTime = 100;
17
18 static EventBarrier* eventBarrier = new EventBarrier("Test EventBarrier");
19

```



```

20 static int *fargv;
21 static int fargc;
22 static int testNum;
23
24 void ThreadInit(int* argv, int argc)
25 {
26     if(argc > 0)
27     {
28         testNum = argv[0];
29         if(argc > 1)
30         {
31             fargv = argv + 1;
32             fargc = argc - 1;
33         }
34         else
35         {
36             fargv = NULL;
37             fargc = 0;
38         }
39     }
40 }
41
42 int getIntBit(int x)
43 {
44     int n = 0;
45     do ++ n, x /= 10; while(x != 0);
46     return n;
47 }
48
49 void makeRandThread(void(*func)(int arg), char *kind, int id, int
50 MinPriority)
51 {
52     int n1 = strlen(kind), n2 = getIntBit(id);
53     char* name = new char[n1 + n2 + 2];
54
55     name[0] = '\0';
56     strcat(name, kind);
57     sprintf(name + n1, " %d", id);
58
59     Thread *thread = new Thread(name, rand() % MinPriority);
60     thread->Fork(func, id);
61

```

```

62     DEBUG('s', "%s: make thread %s %d\n", currentThread->getName(),
63 thread->getName(), thread->getPriority());
64 }
65
66 void makeThread(void(*func)(int arg), char *kind, int id, int priority)
67 {
68     int n1 = strlen(kind), n2 = getIntBit(id);
69     char* name = new char[n1 + n2 + 2];
70
71     name[0] = '\0';
72     strcat(name, kind);
73     sprintf(name + n1, " %d", id);
74
75     Thread *thread = new Thread(name, priority);
76     thread->Fork(func, id);
77
78     DEBUG('s', "%s: make thread %s %d\n", currentThread->getName(),
79 thread->getName(), thread->getPriority());
80 }
81
82 void makeThread(void(*func)(int arg), char *kind, int id)
83 {
84     int n1 = strlen(kind), n2 = getIntBit(id);
85     char* name = new char[n1 + n2 + 2];
86
87     name[0] = '\0';
88     strcat(name, kind);
89     sprintf(name + n1, " %d", id);
90
91     Thread *thread = new Thread(name);
92     thread->Fork(func, id);
93
94     DEBUG('s', "%s: make thread %s\n", currentThread->getName(),
95 thread->getName());
96 }
97
98 void test1_func1(int which)
99 {
100     DEBUG('e', "%s: call Wait.\n", currentThread->getName());
101     eventBarrier->Wait();
102     DEBUG('e', "%s: call Complete.\n", currentThread->getName());
103     eventBarrier->Complete();

```

```
104     DEBUG('e', "%s: finish.\n", currentThread->getName());
105 }
106
107 void test1_func2(int which)
108 {
109     DEBUG('e', "%s: call Signal.\n", currentThread->getName());
110     eventBarrier->Signal();
111     DEBUG('e', "%s: finish.\n", currentThread->getName());
112 }
113
114 void test1_func3(int which)
115 {
116     DEBUG('e', "%s: call Wait.\n", currentThread->getName());
117     eventBarrier->Wait();
118     DEBUG('e', "%s: call Complete.\n", currentThread->getName());
119     eventBarrier->Complete();
120     DEBUG('e', "%s: call Wait.\n", currentThread->getName());
121     eventBarrier->Wait();
122     DEBUG('e', "%s: finish.\n", currentThread->getName());
123 }
124
125 void test1()
126 {
127     makeThread(test1_func2, "Test", 0);
128     makeThread(test1_func3, "Test", 1);
129     makeThread(test1_func3, "Test", 2);
130     makeThread(test1_func1, "Test", 3);
131     makeThread(test1_func2, "Test", 4);
132     makeThread(test1_func3, "Test", 5);
133     makeThread(test1_func1, "Test", 6);
134     makeThread(test1_func1, "Test", 7);
135     makeThread(test1_func3, "Test", 8);
136 }
137
138 void callSignal(int which)
139 {
140     int MaxWaitTime = fargv[2];
141     int waitTime = rand() % MaxWaitTime + 1;
142     alarm->Pause(waitTime);
143     DEBUG('e', "%s: Signal.\n", currentThread->getName());
144     eventBarrier->Signal();
145     DEBUG('e', "%s: finish.\n", currentThread->getName());
```

```

146 }
147
148 void callWait(int which)
149 {
150     int MaxWaitTime = fargv[2];
151     int waitTime = rand() % MaxWaitTime + 1;
152     alarm->Pause(waitTime);
153     DEBUG('e', "%s: call Wait.\n", currentThread->getName());
154     eventBarrier->Wait();
155     DEBUG('e', "%s: call Complete.\n", currentThread->getName());
156     eventBarrier->Complete();
157     DEBUG('e', "%s: finish.\n", currentThread->getName());
158 }
159
160 void test2()
161 {
162     ASSERT(fargc == 3);
163     srand(time(0));
164     int n1 = fargv[0], n2 = fargv[1];
165     for(int i = 0; i < n1; ++ i)
166         makeThread(callSignal, "Signal", i);
167     for(int i = 0; i < n2; ++ i)
168         makeThread(callWait, "Wait", i);
169 }
170
171 void alarmTest(int which)
172 {
173     int maxTime = fargv[1];
174     int howLong = rand() % maxTime + 1;
175     alarm->Pause(howLong);
176 }
177
178 void test3()
179 {
180     ASSERT(fargc == 2);
181     int n = fargv[0];
182     srand(time(0));
183     for(int i = 0; i < n; ++ i)
184         makeThread(alarmTest, "AlarmTest", i);
185 }
186
187 void OneVehicle(int direc)

```

```

188 {
189     DEBUG('e', "%s: arrive %d.\n", currentThread->getName(), direc);
190     ArriveBridge(direc);
191     DEBUG('e', "%s: enter bridge.\n", currentThread->getName());
192     alarm->Pause(CrossBridgeTime);
193     CrossBridge(direc);
194     ExitBridge(direc);
195 }
196
197 void car0(int which)
198 {
199     OneVehicle(0);
200 }
201
202 void car1(int which)
203 {
204     OneVehicle(1);
205 }
206
207 void test4()
208 {
209     makeThread(car0, "Car", 0);
210     makeThread(car0, "Car", 1);
211     makeThread(car1, "Car", 2);
212     makeThread(car0, "Car", 3);
213 }
214
215 void car(int which)
216 {
217     int MaxWaitTime = fargv[1];
218     int waitTime = rand() % MaxWaitTime + 1;
219     alarm->Pause(waitTime);
220     OneVehicle(rand() % 2);
221     DEBUG('e', "%s: \033[40;32mfinished\033[0m.\n",
222 currentThread->getName());
223 }
224
225 void test5()
226 {
227     ASSERT(fargc == 2);
228     srand(time(0));
229     int n = fargv[0];

```

```
230     for(int i = 0; i < n; ++ i)
231         makeThread(car, "Car", i);
232 }
233
234 void Rider(int which)
235 {
236     int numFloor = fargv[0], OtherTime = fargv[4];
237     alarm->Pause(rand() % OtherTime + 1);
238     rider(rand() % numFloor, rand() % numFloor);
239 }
240
241 void test6()
242 {
243     ASSERT(fargc == 5);
244     srand(time(0));
245     int numFloor = fargv[0], numElevator = fargv[1], capacity = fargv[2],
246     numRider = fargv[3];
247     building = new Building("Building", numFloor, numElevator, capacity);
248     for(int i = 0; i < numElevator; ++ i)
249         makeThread(elevator, "Elevator", i);
250     for(int i = 0; i < numRider; ++ i)
251         makeThread(Rider, "Rider", i);
252 }
253
254 void ThreadTest()
255 {
256     switch(testNum)
257     {
258     case 1:
259         test1();
260         break;
261     case 2:
262         test2();
263         break;
264     case 3:
265         test3();
266         break;
267     case 4:
268         test4();
269         break;
270     case 5:
271         test5();
```

```

272         break;
273     case 6:
274         test6();
275         break;
276     default:
277         ASSERT(false);
278         break;
279 }
280 }

```

附件 9 test2 的样例输出

```

1  $ ./nachos -q 2 2 8 1000 -d e
2  Wait 7: call Wait.
3  Signal 0: Signal.
4  Wait 7: call Complete.
5  Wait 2: call Wait.
6  Wait 2: call Complete.
7  Signal 0: finish.
8  Wait 7: finish.
9  Wait 2: finish.
10 Wait 4: call Wait.
11 Wait 1: call Wait.
12 Signal 1: Signal.
13 Wait 4: call Complete.
14 Wait 1: call Complete.
15 Wait 5: call Wait.
16 Wait 5: call Complete.
17 Wait 0: call Wait.
18 Wait 0: call Complete.
19 Signal 1: finish.
20 Wait 4: finish.
21 Wait 1: finish.
22 Wait 5: finish.
23 Wait 0: finish.
24 Wait 6: call Wait.
25 Wait 3: call Wait

```

附件 10 test5 的样例输出

```

1  $ ./nachos -q 5 10 1000 -d e
2  Car 2: arrive 0.
3  Car 2: enter bridge.
4  Car 9: arrive 0.
5  Car 9: enter bridge.
6  Car 7: arrive 1.

```

7 Car 2 cross the bridge, direction:0.
8 Car 2: finished.
9 Car 3: arrive 0.
10 Car 9 cross the bridge, direction:0.
11 Car 9: finished.
12 Car 7: enter bridge.
13 Car 5: arrive 1.
14 Car 6: arrive 0.
15 Car 8: arrive 1.
16 Car 7 cross the bridge, direction:1.
17 Car 7: finished.
18 Car 1: arrive 1.
19 Car 3: enter bridge.
20 Car 0: arrive 0.
21 Car 3 cross the bridge, direction:0.
22 Car 3: finished.
23 Car 5: enter bridge.
24 Car 4: arrive 0.
25 Car 5 cross the bridge, direction:1.
26 Car 5: finished.
27 Car 6: enter bridge.
28 Car 6 cross the bridge, direction:0.
29 Car 6: finished.
30 Car 8: enter bridge.
31 Car 1: enter bridge.
32 Car 8 cross the bridge, direction:1.
33 Car 8: finished.
34 Car 1 cross the bridge, direction:1.
35 Car 1: finished.
36 Car 0: enter bridge.
37 Car 4: enter bridge.
38 Car 0 cross the bridge, direction:0.
39 Car 0: finished.
40 Car 4 cross the bridge, direction:0.
41 Car 4: finished.

附件 11 test6 的样例输出

```
1 $ ./nachos -q 6 10 2 2 10 100 -d e
2 Rider 3: want to go from 0 to 7.
3 Rider 0: want to go from 7 to 4.
4 Rider 1: want to go from 4 to 5.
5 Rider 2: want to go from 5 to 6.
6 Rider 5: want to go from 2 to 7.
```


7 Rider 4: want to go from 3 to 5.
8 Rider 8: want to go from 1 to 0.
9 Elevator 0: open doors.
10 Rider 7: want to go from 6 to 6.
11 Rider 7: finished.
12 Rider 6: want to go from 6 to 4.
13 Rider 9: want to go from 4 to 6.
14 Elevator 1: moving from 0 to 1.
15 Rider 3: enter Elevator 0 in Floor 0.
16 Elevator 0: close doors.
17 Elevator 1: arrive 1.
18 Elevator 1: moving from 1 to 2.
19 Rider 3: request to go to 7.
20 Elevator 1: arrive 2.
21 Elevator 1: open doors.
22 Rider 5: enter Elevator 1 in Floor 2.
23 Elevator 1: close doors.
24 Elevator 1: moving from 2 to 3.
25 Rider 5: request to go to 7.
26 Elevator 1: arrive 3.
27 Elevator 1: open doors.
28 Rider 4: enter Elevator 1 in Floor 3.
29 Elevator 1: close doors.
30 Elevator 1: moving from 3 to 4.
31 Rider 4: request to go to 5.
32 Elevator 1: arrive 4.
33 Elevator 1: open doors.
34 Elevator 1: close doors.
35 Elevator 1: moving from 4 to 5.
36 Elevator 1: arrive 5.
37 Elevator 1: open doors.
38 Rider 4: exit Elevator 1 in Floor 5.
39 Rider 4: finished.
40 Rider 2: enter Elevator 1 in Floor 5.
41 Elevator 1: close doors.
42 Elevator 1: moving from 5 to 6.
43 Rider 2: request to go to 6.
44 Elevator 1: arrive 6.
45 Elevator 1: open doors.
46 Rider 2: exit Elevator 1 in Floor 6.
47 Elevator 1: close doors.
48 Elevator 1: moving from 6 to 7.

49 Rider 2: finished.
50 Elevator 1: arrive 7.
51 Elevator 1: open doors.
52 Rider 5: exit Elevator 1 in Floor 7.
53 Elevator 1: close doors.
54 Rider 5: finished.
55 Elevator 1: moving from 7 to 6.
56 Elevator 1: arrive 6.
57 Elevator 1: moving from 6 to 5.
58 Elevator 1: arrive 5.
59 Elevator 1: moving from 5 to 4.
60 Elevator 1: arrive 4.
61 Elevator 1: moving from 4 to 3.
62 Elevator 1: arrive 3.
63 Elevator 1: moving from 3 to 2.
64 Elevator 1: arrive 2.
65 Elevator 1: moving from 2 to 1.
66 Elevator 1: arrive 1.
67 Elevator 1: open doors.
68 Rider 8: enter Elevator 1 in Floor 1.
69 Elevator 1: close doors.
70 Rider 8: request to go to 0.
71 Elevator 1: moving from 1 to 0.
72 Elevator 1: arrive 0.
73 Elevator 1: open doors.
74 Rider 8: exit Elevator 1 in Floor 0.
75 Elevator 1: close doors.
76 Rider 8: finished.
77 Elevator 1: moving from 0 to 1.
78 Elevator 1: arrive 1.
79 Elevator 1: moving from 1 to 2.
80 Elevator 1: arrive 2.
81 Elevator 1: moving from 2 to 3.
82 Elevator 1: arrive 3.
83 Elevator 1: moving from 3 to 4.
84 Elevator 1: arrive 4.
85 Elevator 1: moving from 4 to 5.
86 Elevator 1: arrive 5.
87 Elevator 1: moving from 5 to 6.
88 Elevator 1: arrive 6.
89 Elevator 1: open doors.
90 Rider 6: enter Elevator 1 in Floor 6.

91 Elevator 1: close doors.
92 Rider 6: request to go to 4.
93 Elevator 1: moving from 6 to 5.
94 Elevator 1: arrive 5.
95 Elevator 1: moving from 5 to 4.
96 Elevator 1: arrive 4.
97 Elevator 1: open doors.
98 Rider 6: exit Elevator 1 in Floor 4.
99 Elevator 1: close doors.
100 Rider 6: finished.
101 Elevator 1: open doors.
102 Rider 1: enter Elevator 1 in Floor 4.
103 Elevator 1: close doors.
104 Rider 1: request to go to 5.
105 Elevator 1: moving from 4 to 5.
106 Elevator 1: arrive 5.
107 Elevator 1: open doors.
108 Rider 1: exit Elevator 1 in Floor 5.
109 Elevator 1: close doors.
110 Rider 1: finished.
111 Elevator 1: moving from 5 to 4.
112 Elevator 1: arrive 4.
113 Elevator 1: open doors.
114 Rider 9: enter Elevator 1 in Floor 4.
115 Elevator 1: close doors.
116 Rider 9: request to go to 6.
117 Elevator 1: moving from 4 to 5.
118 Elevator 1: arrive 5.
119 Elevator 1: moving from 5 to 6.
120 Elevator 1: arrive 6.
121 Elevator 1: open doors.
122 Rider 9: exit Elevator 1 in Floor 6.
123 Elevator 1: close doors.
124 Rider 9: finished.